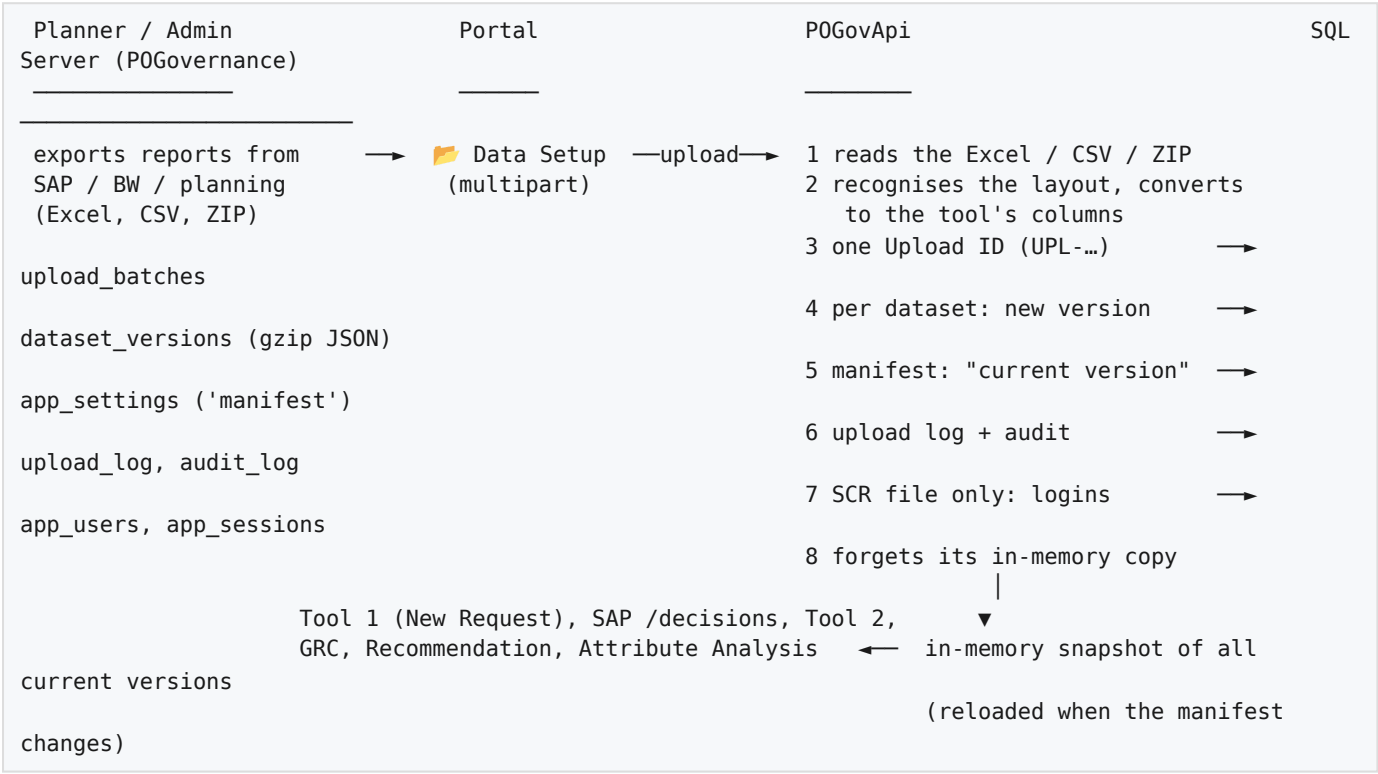
PO Governance — Data Setup: current data source, structure and process

Purpose: a record of how the decision data gets into the tool **today**: which files, where they are stored, in what structure, and which stored procedures run at each step. The next step is to replace the manual files with a real-time source. That change can keep everything described here and only replace **where the rows come from** (section 8).

1. The big picture



Today's data source = files uploaded by people. Nothing is read from SAP, BW or HRMS automatically. The tool keeps **every version** of every file, so any earlier state can be made current again.

2. The eleven datasets

Key	On screen	What it is	Required	Upload
`f1`	F1 — Stock	Closing stock by Brand × MC × MRP × Flag × Colour (+ GM), company level	**yes**	single
`f2`	F2 — Pending PO	Released open PO lines: PO no, GM, colour, delivery month, pending qty, value, vendor	**yes**	single
`f3`	F3 — Sales Trend	Daily sales of the last 45 days by colour (`Day 1` = oldest)	**yes**	single
`f4`	F4 — Forecast	Monthly plan by Brand × MC × MRP × Flag (`MM.YYYY` columns)	**yes**	single
`um`	Universe Master	GMs of the Single Concept MRP universe (sends a GM to engine B)	**yes**	single

Key	On screen	What it is	Required	Upload
`bsr`	BSR — All Seasons	Per GM + colour: weeks on floor, average per-day sell-through %	**yes**	**multi** (files are added together)
`artmaster`	Article Master	GM → brand, division, MC, MRP, flag, season, style, fit, material, design, vendor	no	single
`colmaster`	Colour Master	Colour code → description	no	single
`attrrel`	Attr Relevance Master	Brand × MC × MRP × Flag → assessment dimension	no	single
`catheadmap`	SCR Products / Cat Head Mapping	Brand × MC × MRP → buyer / category-head emails (**creates logins**)	no	single
`grc`	GRC — Buying History	Goods received by Brand × GM × colour per month or week	no (GRC Back Testing only)	own slot

Required: Tool 1 and SAP `/decisions` refuse to decide until F1, F2, F3, F4, Universe Master and BSR are all present.

---

3. Where and how the data is stored (structure)

3.1 Tables

Table	Key	Holds	Written by
`dataset_versions`	(`dkey`, `version`)	**Every version of every dataset**: `data` = gzip of `{"rows":[…],"meta":{…}}`, `bytes`, `created_at`	`PRO_POGOV_BLOB_PUT`
`app_settings`	`name`	Documents as JSON: **`manifest`** (which version is current), **`params`** (planner parameters and seasons), `scr_mapping` (SCR column choices)	`PRO_POGOV_DOC_PUT`
`upload_batches`	`id`	One row per **Upload ID** (`UPL-yyyyMMdd-nnnn`): who, when, source, file names	`PRO_POGOV_LOG_APPEND` (Stream `batches`)
`upload_log`	`id`	One row per dataset change: upload / clear / restore, version, rows, bytes, label, Upload ID	`PRO_POGOV_LOG_APPEND` (Stream `uploads`)
`audit_log`	`id`	One line per change (`upload`, `delete`, `restore`, `restore-upload`, `params`)	`PRO_POGOV_AUDIT APPEND`
`doc_stamps`	`name`	A change number per document (manifest, params, …), increased on every write	inside `PRO_POGOV_DOC_PUT`

3.2 The manifest: "which version is current"

`app_settings`, `name = 'manifest'`:

```
{
  "dataVersion": 57,                // +1 on every upload / clear / restore  → the "New data on the
server" banner
  "paramsVersion": 12,              // +1 on every parameter save
  "datasets": {
    "f1": { "version": "20261103-084245-3fa2", "rows": 18234, "bytes": 912345,
            "label": "F1_Stock.xlsx — 18,234 rows", "updatedBy": "Ravi Kumar", "updatedAt": "2026-
11-03T08:42:45+05:30",
            "uploadId": "UPL-20261103-0001" },
    "f2": { … }, "f3": { … }, …
  }
}
```

3.3 One stored version (`dataset_versions.data`, after gunzip)

```
{ "rows": [ { "Generic Material": "1100123456", "Color": "BLK", "Brand Type": "…", "MC CD": "…",
"MRP": 499,
              "BOGO/NON-BOGO": "NON-BOGO", "OP STK": 120 },
            … ],
  "meta": { "grcPeriodType": "month" } }      // meta only for grc
```

The rows are **exactly the columns of section 7**, already converted from the source reports.

3.4 IDs and formats

Item	Format	Example
Version	`yyyyMMdd-HHmmss-xxxx` (business time, Asia/Kolkata)	`20261103-084245-3fa2`
Upload ID	`UPL-yyyyMMdd-nnnn` (number per day)	`UPL-20261103-0001`
Months	`MM.YYYY`	`11.2026`
Sales days	`Day 1` … `Day 45` (oldest first)	
Kept	every version, forever (`HistoryRetentionDays = 0`)	

---

4. Step by step, with the stored procedures

Notes for all tables in this section:

- **Lock** means `PRO_POGOV_APP_LOCK GET … RELEASE`. Only one writer at a time. It runs on its own connection, ApplicationName `POGovernance (lock)`.
- `PRO_POGOV_` is left out of the procedure names: `DOC_GET` means `PRO_POGOV_DOC_GET`, and so on.

4.1 Opening 📂 Data Setup

#	What	API task	Procedure (@Cmd)
1	Status: data versions, parameters, today's request number, approvals badge	`PO_GET_STATUS`	Lock → `DOC_GET GET_DOC manifest`, `DOC_GET GET_DOC params`, `LOG_READ READ` (requests, today) → release; `LOG_READ DAYS` + `READ_DAYS` (approvals badge)
2	Season pills	`PO_SEASONS_GET`	`DOC_GET GET_DOC manifest` (the seasons come from Article Master in memory)
3	Parameters panel	`PO_PARAMS_GET`	`DOC_GET GET_DOC params`

4.2 Uploading one file into a slot (e.g. F1 — Stock)

#	What happens	Where	Procedure (@Cmd)
1	Planner picks the file	Portal	–
2	File sent to POGovApi	`PO_INGEST_FILE` (multipart, up to `MaxUploadMb` = 400 MB, 900 s)	–
3	Excel / CSV read, sheet found, columns checked and converted	POGovApi (`LegacyImport`)	– (BSR "add": the current BSR is taken from memory and the new file added to it)
4	New **Upload ID**	POGovApi	Lock → `LOG_READ READ` (batches: last number of today) → `LOG_APPEND` (batches) → release
5	**Store the new version**	POGovApi	Lock → `BLOB_PUT` (`ds`, key, version, gzip)

#	What happens	Where	Procedure (@Cmd)
6	**Make it current**		`DOC_GET GET_DOC manifest` → `DOC_PUT manifest` (dataVersion + 1, the new entry)
7	Upload log line		`LOG_APPEND` (uploads: action `upload`) → release
8	Audit line		`AUDIT APPEND` (`upload f1 rows=… bytes=… UPL-…`)
9	Forget the in-memory data	POGovApi	– (the next PO check reloads it, see 4.8)
10	Answer: "Saved to server — Upload ID UPL-..."; the banner on other open pages says "New data on the server"	Portal	–

4.3 ZIP bundle, raw exports, GRC

Path	API task	Differences from 4.2
📦 ZIP bundle	`PO_INGEST_ZIP`	Step 3 recognises every file in the ZIP by name (`F1_Stock…`) or layout. Step 4 runs **once**, and steps 5–8 run **once per dataset found**, all under the same Upload ID.
🧩 Raw export import	`PO_INGEST_RAW`	Step 3 converts SAP/BW report layouts (e.g. stock `Stk q + In Q` → `OP STK`; sales by calendar day → `Day 1..45`; PO `Sum of Pending Qty` → `Pending Qty`). Then as ZIP.
GRC slot	`PO_INGEST_GRC`	Monthly (`Cal. year` row) or weekly (`WW.YYYY`) layout; `meta.grcPeriodType` stored with the rows

4.4 Extra steps for the SCR products file (`catheadmap`)

After 4.2 step 8:

#	What	Procedure (@Cmd)
1	Read the column mapping	`DOC_GET GET_DOC scr_mapping`
2	New buyers / category heads → new logins; extra roles → roles added	`DOC_GET GET_STAMP users` → `DOC_MAP_PUT USERS`
3	Users whose roles grew are signed out (they get the new roles at next sign-in)	`DOC_MAP_PUT SESSIONS`
4	Audit	`AUDIT APPEND` (`scr-sync added=… updated=…`)

4.5 Clearing a file (🗑 on a slot)

#	Procedure (@Cmd)
1	`PO_DATA_CLEAR` → Lock → `DOC_GET GET_DOC manifest` → (new Upload ID: `LOG_READ READ` + `LOG_APPEND` batches) → `LOG_APPEND` (uploads: `clear`) → `DOC_PUT manifest` (key removed, dataVersion + 1) → release → `AUDIT APPEND` (`delete`)

The stored versions stay in `dataset_versions`, so the file can be made current again from History.

4.6 History: make an older version, or a whole upload, current

Action	API task	Procedure (@Cmd)
Open 🗂 History → Data uploads	`PO_HISTORY_UPLOADS`	Lock → `DOC_GET GET_DOC manifest`, `LOG_READ READ` (uploads), `LOG_READ READ` (batches), `BLOB_GET LIST` (ds) → release; `STORE_INFO STORAGE_BYTES`

Action	API task	Procedure (@Cmd)
Make one version current	`PO_DATA_RESTORE`	Lock → `LOG_READ READ` (uploads) → `BLOB_GET SIZE` (does it still exist?) → `DOC_GET manifest` → `DOC_PUT manifest` → `LOG_APPEND` (uploads: `restore`) → release → `AUDIT APPEND` (`restore`)
Make a whole upload current	`PO_UPLOAD_RESTORE`	As above for every dataset of that Upload ID, under a new Upload ID → `AUDIT APPEND` (`restore-upload`)
Download a stored version / upload as Excel	`PO_HISTORY_EXPORT_DATA` / `PO_HISTORY_EXPORT_UPLOAD`	`BLOB_GET GET` (per dataset)

4.7 Parameters and seasons

Action	API task	Procedure (@Cmd)
Save parameters (T1, T2, mix, N, WOF, seasons …)	`PO_PARAMS_SAVE`	Lock → `DOC_PUT params` → `DOC_GET manifest` → `DOC_PUT manifest` (paramsVersion + 1) → release → `AUDIT APPEND` (`params`)
Click a season pill	`PO_SEASONS_TOGGLE`, then `PO_PARAMS_SAVE`	Toggle: `DOC_GET GET_DOC manifest` only (worked out in memory); then the portal saves seasonCurrent / seasonFallback as **Save parameters**

4.8 How the engines read the data

When	Procedure (@Cmd)	What
POGovApi start (once, background)	`DOC_GET GET_DOC manifest` → `BLOB_GET GET` × every current dataset → `DOC_GET GET_DOC params`	All current files into memory before the first PO check. Today this takes 20–25 s when the API runs far from the database (f3 ≈ 7 s, artmaster ≈ 8 s, about 15 MB).
Every PO check / SAP decision / Tool 2 run	`DOC_GET GET_DOC manifest`	"Same dataVersion and paramsVersion as in memory?" Yes → no more SQL.
After an upload / clear / restore / parameter save	`DOC_GET GET_DOC manifest` → `BLOB_GET GET` **only for the datasets whose version changed** → `DOC_GET GET_DOC params`	The new snapshot replaces the old one

4.9 MRA Status (morning file + SAP during the day)

A card on 📂 Data Setup, separate from the eleven datasets: the MRA lines are stored as **table rows** (`SAP_MRA_STATUS`), not as a gzipped version, because SAP changes single lines all day. SQL Server only.

Step	Procedure (@Cmd)	What
Morning: ⬆ Upload MRA file (.xlsx / .csv)	– (read in the API)	Header row found by name (MRA No / MRN No, Qty / PO Qty, Status / Decision …). **MRA No, Qty and Status are required**; one wrong row → the whole file is refused with the row numbers, nothing changes
	`PRO_SAP_MRA_STATUS UPLOAD`	One transaction: **empties** `SAP_MRA_STATUS`, stores the file's lines, logs the upload in `SAP_MRA_UPLOADS`
During the day: SAP `POST /api/sap/v1/mra-status`	`PRO_SAP_MRA_STATUS APPLY`	Each change in order: CREATED / UPDATED / UNCHANGED / STALE / AMBIGUOUS; every change written to `SAP_MRA_STATUS_HISTORY`
Opening the card, ↻ Refresh	`PRO_SAP_MRA_STATUS SUMMARY` → `LIST`	Lines / MRAs / quantity per status, last upload, SAP changes today; the list with search and paging
Clicking an MRA	`PRO_SAP_MRA_STATUS HISTORY`	Every SAP change of that MRA

Statuses: ALLOW, WATCHLIST, CANCELLED, DISCARDED, REJECTED, SELF_REJECTED (setting `SapInterface:MraStatuses`). Full SAP contract: `docs/sap/SAP-API-Reference.md` §8.

5. Who can do what

Action	Roles
Upload, clear, restore, parameters, seasons	Admin, Planner
See Data Setup and History → Data uploads	Admin, Planner
Use the data (Tool 1 New Request)	everyone signed in

6. Procedure index for Data Setup

Procedure	@Cmd	Used in
`PRO_POGOV_APP_LOCK`	`GET` / `RELEASE`	every write (4.2–4.7), status
`PRO_POGOV_BLOB_PUT`	–	new version (4.2 step 5)
`PRO_POGOV_BLOB_GET`	`GET` / `SIZE` / `LIST`	engines (4.8), restore (4.6), history, exports
`PRO_POGOV_DOC_GET`	`GET_DOC manifest / params / scr_mapping`	every step that needs "what is current"
`PRO_POGOV_DOC_PUT`	manifest / params	make current (4.2, 4.5, 4.6, 4.7)
`PRO_POGOV_LOG_READ`	`READ` (batches, uploads)	next Upload ID, history, restore
`PRO_POGOV_LOG_APPEND`	batches / uploads	Upload ID, upload log
`PRO_POGOV_AUDIT`	`APPEND`	audit lines
`PRO_POGOV_DOC_MAP_PUT`	`USERS` / `SESSIONS`	SCR file only (4.4)
`PRO_POGOV_STORE_INFO`	`STORAGE_BYTES`	History page total
`PRO_SAP_MRA_STATUS`	`UPLOAD` / `APPLY` / `LIST` / `SUMMARY` / `HISTORY`	MRA Status card and SAP `mra-status` (4.9)

7. Columns the tool reads, per dataset (the structure to keep)

Header matching for most engines ignores case and spaces. The Recommendation, Attribute Analysis, GRC and Cat-Head lookups need the **exact** header, so the importer always writes the first name listed.

Dataset	Columns (first name = the one the importer writes; alternates accepted today)
f1	`Generic Material` · `Color` (Colour) · `Brand Type` · `DIV` (Division) · `MC CD` · `MC DESC` · `MRP` · `BOGO/NON-BOGO` (Flag, Article Type, Flag (BOGO/NON-BOGO)) · `OP STK` (Op Stk)
f2	`PO Number` (PO No, Purchase Order …) · `Generic Material` · `Color` · `Brand Type` · `DIV` · `MC CD` · `MC DESC` · `MRP` · `BOGO/NON-BOGO` · `Delivery Month` (`MM.YYYY`; DEL MONTH) · `Pending Qty` (Released Qty) · `PO Value` (PO Val, Value, PO Amount) · `Vendor Name` (Vendor) · `Season` · `ART DESC`
f3	`Generic Material` · `Color` · `Brand Type` · `MC CD` · `MRP` · `BOGO/NON-BOGO` · `Day 1` … `Day N` (`D N`; oldest first) — or `MM.YYYY` month columns / `CY APR & MAY AVG`
f4	`Brand Type` · `MC CD` · `MRP` · `BOGO/NON-BOGO` · `MM.YYYY` month columns
um	`Generic Material` · `Brand Type` · `Division` · `MC CD` · `MC DES` · `MRP` · `Flag (BOGO/NON-BOGO)`
bsr	`Generic Material` · `Color` · `MC CD` · `MRP` · `Article Type` (BOGO/NON-BOGO, Flag) · `WEEK ON FLOOR` (WOF) · `AVG PERDAY SELL THRU%`
artmaster	`Generic Material` · `ART DESC` · `Brand Type` · `Division` · `MC CD` · `MC DESC` · `MRP` · `Flag (BOGO/NON-BOGO)` · `Season` · `Style` · `Fit` · `Product Material` · `Design` · `Vendor` · `VENDOR NAME` (any other column = an attribute in Attribute Analysis)

Dataset	Columns (first name = the one the importer writes; alternates accepted today)
colmaster	`Colour` (Color) · `Colour Desc` (Color Desc, Description, Desc)
attrrel	`Brand Type` · `MC CD` · `MRP` · `BOGO/NON-BOGO` · `Assessment Dimension`
catheadmap	`Brand Type` · `MC CD` · `MRP` · buyer / category-head email columns (Level 1/2 → buyer, Level 3 → category head; `Category_Email_ID` for Tool 2 / GRC)
grc	own layout: `Brand Type`, `Generic Material`, art description, `Colour`; monthly `MM.YYYY` under a `Cal. year` row or weekly `WW.YYYY` with `GRC Q` / `GRC V`

Formats: delivery and plan months `MM.YYYY`; day columns `Day N`; numbers as plain numbers; per-day sell-through as a percentage.

---

8. Moving to a real-time data source while keeping this structure

8.1 What stays exactly as it is

- The **dataset keys** (`f1` … `grc`) and the **column names and formats** of section 7.
- The **storage**: `dataset_versions` (one new version per refresh), `manifest`, `upload_batches`, `upload_log`, `audit_log`.
- The **engines** (Tool 1, SAP decisions, Tool 2, GRC, Recommendation), History, restore and audit. They only read the current versions, so they don't notice where the rows came from.

8.2 What changes

Only **steps 1–3 of 4.2**: instead of a person uploading a file that is parsed, a connector reads the source (SQL Server linked server or API, see `docs/data-automation/Data-Setup-Automation-Design.md`) and produces the **same rows**. From step 4 on, the existing code is reused unchanged:

```
 real-time source ──► connector: rows with the section-7 columns ──►
DataService.SaveVersionAsync(key, rows, label, uploadId)
                                                                    = 4.2 steps 4–9 (same
procedures, same tables)
```

8.3 Two ways to plug in

	How	Effort
A. Inside POGovApi (recommended)	A background job reads the source on a schedule, builds the rows and calls `NewUploadBatchAsync` + `SaveVersionAsync` for each dataset, with the source `auto-pull` and the snapshot time as label	The design document's phase 1
B. From outside, available today	A job signed in as a planner or admin calls `PO_UPLOAD_START` (new Upload ID), then `PO_DATA_PUT` per dataset: multipart with the file `{"rows":[…]}` (plain or gzip) and the fields `Key`, `Rows`, `Label`, `UploadId`	No change to the tool. Good for a first test with real data.

8.4 Checklist before switching a dataset

1. For each dataset, map every column of section 7 to a source field. Write down how the units are converted (dates to `MM.YYYY`, sales per day to `Day 1..45`, stock including in-transit, …).
2. Pull once and **compare** with the latest uploaded version: row count, quantity total, and a sample of Tool 1 and SAP decisions.
3. Keep the **six required datasets** consistent: refresh them together, from the same snapshot.
4. Keep the manual upload available until the comparison has been good for a few days. Every version stays in History, so any upload can be made current again.